Px GYM

Roteiro de apresentação

Fase “Domínio e Interface Primeiro”, 100% mock, com mais de 80% de cobertura. Guia completo para apresentar o deck de 27 slides: preparação, tempo, o que falar em cada slide, onde apontar e como responder às perguntas.

Paulo Rogério · Sistemas de Informação · CEFET-MG · 2026

Parte 1 — Antes de apresentar

Leia esta parte com calma pelo menos um dia antes. Ela cobre a mensagem, o tempo, a preparação do computador e a forma de usar as referências de código, que foi o pedido explícito do professor.

Como usar este roteiro

- **Parte 1** prepara você: mensagem, tempo, computador, teclas e demonstração.
- **Parte 2** é o roteiro slide a slide, com miniatura, tempo, fala sugerida, onde apontar e perguntas prováveis.
- **Parte 3** reúne as perguntas difíceis com respostas prontas.
- **Parte 4** é material de consulta rápida: mapa de onde está cada coisa, glossário, o que não dizer e o checklist do dia.

A fala sugerida não é para decorar nem para ler. Cada parágrafo carrega uma ideia; entenda a ideia e diga do seu jeito. O que vale a pena decorar são só três coisas: a frase de abertura, as duas invariantes do slide 7 e a frase final.

A mensagem em uma frase

Se a regra de negócio mora num lugar que não conhece tela nem banco, dá para provar que ela está certa antes de existir tela ou banco.

Tudo o que você mostrar é evidência dessa frase. Se der branco em qualquer momento, volte para ela.

As três ideias que o professor precisa levar

1. **A arquitetura respeita a direção de dependência.** O domínio não importa ninguém; a Context API fica na borda; um único arquivo conhece classes concretas.
2. **As regras estão no domínio e são testadas pelo nome.** As duas invariantes existem no agregado, com teste que leva a palavra “INVARIANTE”.
3. **Está tudo em mock e medido.** Sem SQLite, sem Supabase, 416 testes passando, 93% de cobertura no core e 99% nos adapters, com o limite de 80% configurado no Jest.

O que o professor pediu e onde isso aparece

SEÇÃO	SLIDES	O PONTO QUE NÃO PODE FALTAR
1 · Domínio puro	5-9	Value objects imutáveis; agregado com as duas invariantes; duas entidades; três serviços; contratos como interfaces
2 · Aplicação	10-13	Os dez casos de uso; orquestram sem decidir regra; fakes em Map; teste de caminho feliz, erro e permissão
3 · Adapters e Context	14-18	AuthProvider na raiz, sem prop drilling; sessão segura no SecureStore; hooks com idle, salvando e salvo; CameraGatewayExpo
4 · Interface e rotas	19-20	Expo Router como fronteira; View e Text com StyleSheet e Flexbox; as três telas nomeadas
5 · Mock	21	Sem banco, sem sensor real em teste, repositórios em Map, stub de sessão
6 · Testes e cobertura	22-24	Pirâmide com números; meta de 80%; RNTL com render, press e changeText; um E2E
7 · Checklist	25-27	Os oito passos entregues; limites ditos com franqueza

Plano de tempo

No ritmo sugerido, a apresentação leva cerca de **21 minutos**, sem contar perguntas. A coluna “no relógio” mostra em que minuto você deve estar ao terminar cada seção. Deixe o cronômetro do celular visível.

SEÇÃO	SLIDES	DURAÇÃO	NO RELÓGIO
Abertura	1-4	2:25	0:00 → 2:25
1 · Domínio puro	5-9	4:45	2:25 → 7:10
2 · Aplicação	10-13	3:25	7:10 → 10:35
3 · Adapters e Context	14-18	3:40	10:35 → 14:15
4 · Interface e rotas	19-20	1:30	14:15 → 15:45
5 · O que fica em mock	21	0:50	15:45 → 16:35
6 · Testes e cobertura	22-24	2:20	16:35 → 18:55
7 · Checklist e fecho	25-27	2:15	18:55 → 21:10
Total	**1-27**	**21:10**	

Se você tiver 15 minutos

Fale normalmente os slides 1 a 7, 10, 11, 13, 14, 16, 20, 21, 22 e 25 a 27. Passe em 15 a 20 segundos, com uma frase só, pelos slides 8, 9, 12, 17, 18, 19, 23 e 24. Pule o 15.

Se você tiver 10 minutos

Caminho essencial: 1, 3, 4, 6, 7, 10, 11, 14, 16, 20, 21, 22, 26 e 27. Nos slides 4, 10, 16 e 20 fale só a primeira frase. Os outros slides você atravessa sem parar, dizendo o título. Cada bloco da Parte 2 tem a linha “se o tempo estiver curto” com a decisão para aquele slide.

Se estourar o tempo no meio: não acelere a fala. Corte slides. Um slide bem explicado vale mais do que três atropelados, e os que não podem ser cortados são o 7 (invariantes), o 21 (mock), o 22 (pirâmide) e o 26 (limites).

Preparação na véspera

- [ ] Abra o deck pela pasta certa: `C:\dev\Px GYM\docs\apresentacao\apresentacao-dominio-interface.html`. A pasta antiga do OneDrive está vazia.
- [ ] Passe os 27 slides uma vez em tela cheia, no monitor ou projetor que vai usar, e veja se o código está legível do fundo da sala.
- [ ] Abra também a versão completa, `apresentacao-dominio-interface-completa.html`, em uma segunda aba. É nela que estão os trechos de código de cada item.
- [ ] Gere um PDF de reserva do deck: `Ctrl` + `P`, paisagem, margens “nenhuma” e a opção “gráficos de plano de fundo” marcada.
- [ ] Rode os testes uma vez, para aquecer o cache e confirmar que está tudo verde (comandos na seção de demonstração).
- [ ] Ensaie em voz alta, com cronômetro, pelo menos duas vezes. Na primeira você vai passar do tempo; é normal.
- [ ] Copie a pasta do projeto ou os dois arquivos HTML para um pendrive. Os decks funcionam sem internet.

Quinze minutos antes

- [ ] Notebook na tomada, modo “não perturbe” ligado, notificações e abas pessoais fechadas.
- [ ] Navegador com zoom em 100% (`Ctrl` + `0`). O deck se ajusta sozinho à tela.
- [ ] Aba 1: deck de 27 slides, no slide 1, em tela cheia. Aba 2: deck completo.
- [ ] Editor de código aberto na pasta `C:\dev\Px GYM`, com o arquivo `ciclo-avaliacao.ts` já aberto.
- [ ] Terminal aberto em `C:\dev\Px GYM`, com fonte grande, caso peçam para rodar os testes.
- [ ] Cronômetro do celular à vista, água por perto, este roteiro impresso ou no celular.

Teclas do deck

TECLA	O QUE FAZ
`→` `espaço` `PageDown`	Próximo slide. Passador de slides sem fio costuma enviar PageDown e funciona.
`←` `PageUp`	Slide anterior
`Home` / `End`	Primeiro / último slide
`O`	Visão geral com todos os slides. Clique em um para ir direto. É o jeito mais rápido de voltar a um slide durante as perguntas.
`F`	Tela cheia
Clique	Na metade direita avança, na esquerda volta. **Cuidado:** clicar para “mostrar” algo com o mouse também troca de slide. Aponte movendo o cursor, sem clicar.

O endereço guarda o slide atual. Para pular direto, troque o número no fim do endereço, por exemplo `…interface.html#17`, e pressione Enter. Os itens da agenda (slide 2) também são clicáveis.

Como usar as referências de código

O professor disse que quer ver a referência aos códigos. Em quase todo slide há uma faixa “no código” no rodapé, ou chips dentro das tabelas, com o arquivo e a linha.

- **Como ler:** `core/` é `packages/core/src/`; `mobile/` é `apps/mobile/src/`; `raiz/` é a raiz do repositório. O número em verde é a linha.
- **Como falar:** não leia os chips. Diga uma vez por seção algo como “o arquivo e a linha estão aqui embaixo, se o senhor quiser que eu abra”. Nos slides com código, aponte a linha destacada e diga o que ela faz.
- **Se pedirem para abrir um arquivo:** no VS Code, `Ctrl`+`P`, digite o nome do arquivo seguido de dois pontos e a linha, por exemplo `ciclo-avaliacao.ts:390`, e Enter. Em outro editor, abra o arquivo e use “ir para a linha”.
- **Se pedirem o código de um item que só aparece em tabela:** vá para a aba do deck completo. A Parte 4 deste roteiro tem o mapa de qual slide abrir.
- **Os números de linha são reais.** O deck é gerado por um script que procura cada trecho no código; se você alterar o código, gere o deck de novo antes de apresentar (`node docs/apresentacao/dominio-interface/build.mjs`, a partir de `C:\dev\Px GYM`).

Demonstração ao vivo (opcional)

Não é obrigatória, e só vale a pena se sobrar tempo ou se pedirem. A melhor demonstração é rodar os testes: mostra em poucos segundos que os números dos slides são reais.

Core: domínio e casos de uso, com a tabela de cobertura

```
cd "C:\dev\Px GYM\packages\core"
npm run test:coverage
```

Leva cerca de meio minuto. O que mostrar na saída: “Tests: 281 passed”, a linha “All files” da tabela com 93% e as linhas dos arquivos do ciclo em 100%.

Mobile: Context, hooks, telas e o teste de ponta a ponta

```
cd "C:\dev\Px GYM\apps\mobile"
npm test
```

Leva de dez segundos a um minuto, conforme o cache. O que mostrar: “Tests: 115 passed” e a linha do arquivo `fluxo-relatorio.e2e.test.tsx`.

- **Rode antes, no mesmo dia.** A primeira execução é a mais lenta. E rode sempre a partir de `C:\dev`; na pasta do OneDrive o Jest não encontrava os testes.
- **Enquanto roda, fale.** Explique o que está sendo executado em vez de ficar olhando o terminal em silêncio.
- **Se falhar ao vivo:** não tente consertar. Diga “rodei hoje mais cedo e estava verde; mostro o resultado depois” e volte ao deck.
- **Não tente rodar o app no emulador** durante a apresentação. Esse fluxo não foi validado em aparelho nesta fase, e você mesmo diz isso no slide 26.

Como falar

- **Abra cada seção dizendo o número dela:** “seção dois, a camada de aplicação”. O professor está conferindo uma lista.

- **Nos slides de código, não leia o código.** Aponte a linha destacada e diga a regra em português.
- **Nos slides de tabela, não leia a tabela.** Diga o que ela prova e leia no máximo uma linha como exemplo.
- **Fale para as pessoas, não para a tela.** Olhe para o slide só para apontar.
- **Diga os números devagar.** “Quatrocentos e dezesseis testes” é uma frase inteira; faça uma pausa depois.
- **Tratamento:** a fala sugerida usa “o senhor”. Troque pelo tratamento que você já usa com o professor.

Parte 2 — Roteiro slide a slide

Cada bloco traz a miniatura do slide, o tempo, o que falar, onde apontar e as perguntas que podem surgir naquele ponto. O texto de “o que falar” é para ser estudado, não lido: entenda a ideia de cada parágrafo e diga com as suas palavras.



Slide 1 de 27

Capa

20 s · no relógio: 0:00 → 0:20

Objetivo. Dizer em uma frase o que é a entrega, antes de qualquer detalhe.

O QUE FALAR

Bom dia. Esta é a fase “Domínio e Interface Primeiro” do Px GYM. A ideia da fase é construir e provar as regras de negócio e as telas antes de existir banco de dados: tudo roda em mock, com mais de 80% de cobertura de testes.

A ordem de construção foi de dentro para fora: domínio, casos de uso, Context API com sessão segura e, por último, as telas.

ONDE APONTAR

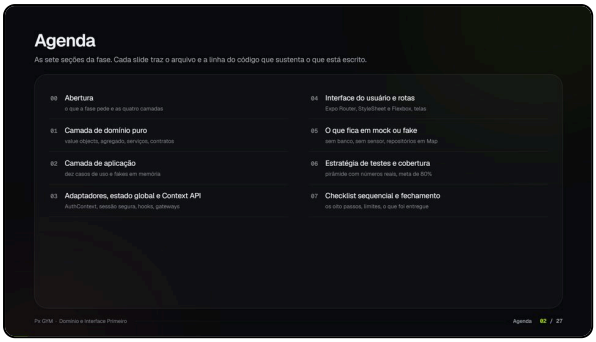
- Nada para apontar. Olhe para a turma, não para a tela.

SE O TEMPO ESTIVER CURTO

Mantenha. São 20 segundos.

FRASE DE PASSAGEM

“A apresentação segue as sete seções da fase.”



Slide 2 de 27

Agenda

20 s · no relógio: 0:20 → 0:40

Objetivo. Mostrar as sete seções da fase e avisar que todo slide tem referência de código.

O QUE FALAR

A apresentação tem sete seções, que são as sete partes da fase.

Um aviso que vale para todos os slides: no rodapé de cada um aparece o arquivo e a linha do código que sustenta o que está escrito. Se o senhor quiser ver qualquer trecho, é só pedir que eu abro.

ONDE APONTAR

- A lista das sete seções.
- Diga a frase sobre as referências olhando para o professor: foi um pedido dele.

SE O TEMPO ESTIVER CURTO

Mantenha, mas não leia os itens um a um.

FRASE DE PASSAGEM

“Antes de entrar no código, o que a fase pede.”



Slide 3 de 27

O que esta fase pede

60 s · no relógio: 0:40 → 1:40

Objetivo. Fixar o fluxo de dentro para fora e dar os cinco números que resumem a entrega.

O QUE FALAR

O fluxo é incremental e de dentro para fora. Primeiro o domínio, que é TypeScript puro. Depois os casos de uso, que só orquestram. Depois a camada de adapters, onde ficam a Context API, a sessão segura e os hooks. Por último as telas, testadas com casos de uso falsos injetados.

Cada etapa só começa quando a anterior já tem teste passando, e nenhuma delas conhece banco de dados.

Os números: 416 testes passando; 93% de cobertura de linhas no core, que é domínio mais aplicação; 99% na camada de adapters; dez casos de uso; e zero conexão com banco no modo mock.

ONDE APONTAR

- As quatro caixas, da esquerda para a direita, acompanhando a fala.
- A faixa de números. Pare um segundo em “93%” e em “0”.

SE O TEMPO ESTIVER CURTO

Mantenha. É o slide que dá o placar.

FRASE DE PASSAGEM

“Antes de mostrar as classes, a arquitetura em que elas se encaixam.”

PERGUNTAS QUE PODEM VIR AQUI

Por que 416 e não só os testes desta fase?

Porque a suíte inteira roda junta. Só desta fase são 248. O restante são gateways, memória, modo mock e os testes do app que já existia, que continuam passando.

Onde mostrar: slide 25 deste deck (checklist)

A menor métrica de cobertura é quanto?

87%, em funções do core. O que puxa para baixo são entidades antigas do app, sem teste próprio, que esta fase não tocou. Nos arquivos do ciclo de avaliação a cobertura é praticamente total.

Onde mostrar: slide 23 (cobertura)



Slide 4 de 27

Quatro camadas, uma direção de dependência

45 s · no relógio: 1:40 → 2:25

Objetivo. Mostrar a regra de dependência e ensinar a ler os chips de referência.

O QUE FALAR

São quatro camadas e a dependência só aponta para a direita: rotas dependem de adapters, que dependem da aplicação, que depende do domínio. O domínio não importa ninguém.

A prova está no canto direito: esses são os imports do agregado principal. Só erros e value objects do próprio domínio. Uma busca por React, Expo, Supabase ou Drizzle nas pastas de domínio e aplicação devolve zero ocorrências.

E como ler as referências: “core” é a pasta packages/core/src, “mobile” é apps/mobile/src, e o número em verde é a linha.

ONDE APONTAR

- As setas entre as caixas.
- O bloco de código com os seis imports.
- Um chip do rodapé, mostrando caminho e linha.

SE O TEMPO ESTIVER CURTO

Mantenha, mas pode falar só o primeiro parágrafo.

FRASE DE PASSAGEM

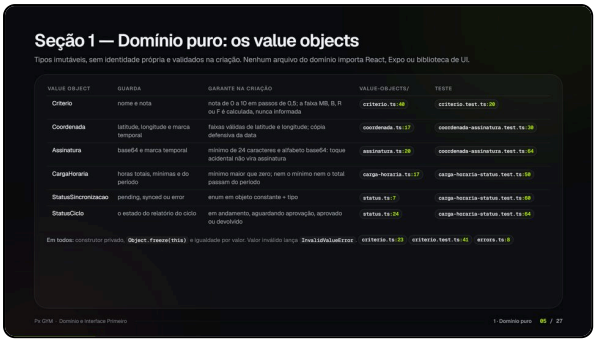
“Começando pela camada mais de dentro: o domínio.”

PERGUNTAS QUE PODEM VIR AQUI

Como você garante que ninguém importa React no domínio?

Hoje pela estrutura de pacotes: o core é um pacote separado que não tem React como dependência, e os testes dele rodam em ambiente Node puro. Uma regra de lint seria o próximo passo para tornar isso automático.

Onde mostrar: completa, slide 5



Slide 5 de 27

Seção 1 — Domínio puro: os value objects

60 s · no relógio: 2:25 → 3:25

Objetivo. Cobrir os seis value objects e as três propriedades de um value object: imutável, sem identidade, validado na criação.

O QUE FALAR

Value objects são tipos imutáveis, sem identidade própria e validados na criação. São seis.

Criterio guarda nome e nota de zero a dez, em passos de meio ponto, e calcula a faixa: muito bom, bom, regular ou fraco. A faixa é sempre calculada, nunca informada, para que nota e faixa não possam discordar.

Coordenada guarda latitude, longitude e marca temporal. Assinatura guarda o base64 e a marca temporal, e recusa um toque acidental no quadro. CargaHoraria valida horas totais, mínimas e do período. E os dois enums: StatusSincronizacao e StatusCiclo, que é o estado do relatório.

Em todos eles o construtor é privado, o objeto é congelado com Object.freeze e a igualdade é por valor.

ONDE APONTAR

- A linha do Criterio, coluna “Garante na criação”.
- A coluna de testes: cada value object tem o seu.
- A nota de baixo com Object.freeze.

SE O TEMPO ESTIVER CURTO

Fale só Criterio como exemplo e a frase das três propriedades.

FRASE DE PASSAGEM

“Esses valores são as peças. Quem monta as peças e protege as regras é o agregado.”

PERGUNTAS QUE PODEM VIR AQUI

O que diferencia um value object de uma entidade?

Identidade. Dois critérios com o mesmo nome e a mesma nota são o mesmo critério; não existe id. Uma entidade continua sendo ela mesma ainda que seus dados mudem, porque tem identidade.

Onde mostrar: completa, slide 12

Como você prova que é imutável?

Tem teste: ele confere que o objeto está congelado e que tentar alterar a nota lança TypeError.

Onde mostrar: completa, slide 12 (criterio.test.ts, “é imutável”)

Posso ver o código do Criterio?

Sim. Na versão completa, slide 7, está o método criar e o cálculo da faixa com as linhas destacadas.

Onde mostrar: completa, slides 7 a 11



Slide 6 de 27

Aggregate Root: CicloAvaliacao

60 s · no relógio: 3:25 → 4:25

Objetivo. Explicar o que o agregado agrupa e o ciclo de vida do relatório, usando o diagrama.

O QUE FALAR

O agregado raiz é o CicloAvaliacao: o relatório de um ciclo de treino. Ele agrupa as atividades desenvolvidas, a avaliação do personal, a autoavaliação e as assinaturas, e controla o ciclo de vida do relatório.

O estado é privado: nada muda por fora, só por método do agregado. Por isso ele é o único lugar que conhece as invariantes.

O ciclo de vida está no diagrama. O relatório nasce em andamento. Quando aluno e personal assinam, ele vai sozinho para aguardando aprovação. A coordenação aprova ou devolve. Se devolver, as assinaturas são apagadas, porque o conteúdo vai mudar, e o ciclo reabre para correção.

ONDE APONTAR

- A tabela da esquerda: o que o agregado guarda.
- No diagrama, siga o caminho com o dedo ou o cursor: em_andamento → aguardando → aprovado; depois a volta por devolvido.

SE O TEMPO ESTIVER CURTO

Mantenha. O diagrama explica metade da fase.

FRASE DE PASSAGEM

"O agregado protege duas invariantes. As duas estão no próximo slide, em código."

PERGUNTAS QUE PODEM VIR AQUI

Por que devolver apaga as assinaturas?

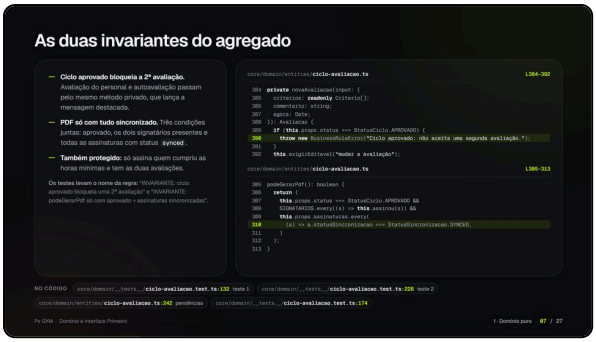
Porque a assinatura atesta um conteúdo. Se o relatório volta para correção, o conteúdo vai mudar, então a assinatura antiga não vale mais. Os dois precisam assinar de novo.

Onde mostrar: completa, slide 15

Quais são as regras de um treino dentro do ciclo?

Data dentro do período do ciclo, duração de até quatro horas, não passar do teto de horas do período, e só enquanto ninguém assinou.

Onde mostrar: completa, slide 17



Slide 7 de 27

As duas invariantes do agregado

75 s · no relógio: 4:25 → 5:40

Objetivo. Este é o slide mais importante do domínio. Mostrar as duas regras que o agregado protege, no código, com o teste que as prova.

O QUE FALAR

O agregado protege duas invariantes. As duas estão aqui.

Primeira: bloquear a segunda avaliação se o período já estiver aprovado. Tanto a avaliação do personal quanto a autoavaliação passam por este método privado, novaAvaliacao. Na linha destacada, se o status é aprovado, ele lança um erro de regra de negócio com a mensagem “ciclo aprovado: não aceita uma segunda avaliação”.

Segunda: podeGerarPdf só com todas as assinaturas sincronizadas. São três condições juntas: status aprovado, os dois signatários presentes e, na linha destacada, toda assinatura com status synced.

Os testes dessas duas regras levam a palavra “INVARIANTE” no nome, para ficarem fáceis de achar.

ONDE APONTAR

- A linha destacada do primeiro bloco (o throw).
- A linha destacada do segundo bloco (statusSincronizacao === SYNCED).
- Os dois primeiros chips do rodapé: são os testes.

SE O TEMPO ESTIVER CURTO

Não corte. Se sobrar só um slide de domínio, é este.

FRASE DE PASSAGEM

“Além do agregado, o domínio tem duas entidades com raiz própria.”

PERGUNTAS QUE PODEM VIR AQUI

E se a assinatura falhar no envio?

Ela fica com status error, que não é synced, então o PDF continua bloqueado. Tem teste específico para isso.

Onde mostrar: completa, slide 16

Onde essa regra é usada na tela?

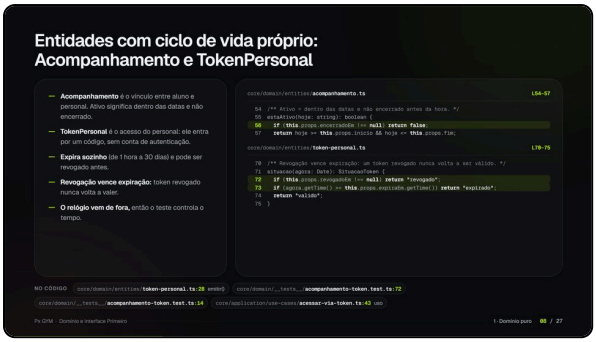
O botão “Gerar PDF” fica desabilitado enquanto a regra responde não, e a tela lista o que falta. A tela não calcula nada: ela lê a resposta que o domínio já deu.

Onde mostrar: completa, slide 64

Pode abrir esse arquivo?

Sim: packages/core/src/domain/entities/ciclo-avaliacao.ts, linha 390 para a primeira regra e 305 para a segunda.

Onde mostrar: abrir no editor



Slide 8 de 27

Entidades: Acompanhamento e TokenPersonal

45 s · no relógio: 5:40 → 6:25

Objetivo. Mostrar as duas entidades com ciclo de vida próprio e o acesso do personal sem conta.

O QUE FALAR

Acompanhamento é o vínculo entre aluno e personal. Ele está ativo quando está dentro das datas e não foi encerrado.

TokenPersonal é o acesso do personal. Ele entra por um código, sem ter conta de autenticação. O token expira sozinho e pode ser revogado antes. E a revogação vence a expiração: um token revogado nunca volta a valer.

Repare que o método situacao recebe a data como parâmetro. O relógio vem de fora, e é isso que deixa o teste controlar o tempo.

ONDE APONTAR

- No segundo bloco, as duas linhas destacadas: revogado antes de expirado.

SE O TEMPO ESTIVER CURTO

Uma frase para cada entidade e siga.

FRASE DE PASSAGEM

“Faltam as regras que cruzam entidades e os contratos.”

PERGUNTAS QUE PODEM VIR AQUI

Por que o personal não tem conta?

Porque ele não precisa de cadastro para avaliar um aluno: o acesso é por token, com expiração e revogação. O aluno gera o código e o personal entra com ele. A sessão criada expira junto com o token.

Onde mostrar: completa, slides 19 e 35



Slide 9 de 27

Serviços de domínio e contratos

45 s · no relógio: 6:25 → 7:10

Objetivo. Fechar a seção 1: os três serviços e as interfaces declaradas no domínio.

O QUE FALAR

Os serviços de domínio guardam regras que cruzam entidades. RegraGeracaoPdfService diz se o PDF pode sair e, se não pode, o que falta. RegraDevolucaoService diz quem pode devolver, quando e com que justificativa. SincronizacaoService decide a ordem da fila: assinatura primeiro, porque é ela que destrava o PDF.

Eles não têm estado nem entrada e saída: recebem o agregado e devolvem uma resposta.

À direita estão os contratos. São interfaces puramente declarativas: repositórios, CameraGateway, LocationGateway, AuthGateway. O domínio declara do que precisa; quem implementa é a borda.

ONDE APONTAR

- As três linhas de serviço.
- Na tabela de contratos: CameraGateway, LocationGateway e AuthGateway.

SE O TEMPO ESTIVER CURTO

Leia só os nomes dos três serviços e diga “contratos são só interfaces”.

FRASE DE PASSAGEM

“Isso fecha o domínio. Agora, quem usa o domínio: a camada de aplicação.”

PERGUNTAS QUE PODEM VIR AQUI

Por que a ordem da fila é regra de domínio e não detalhe técnico?

Porque ela tem consequência de negócio: o PDF só é liberado com as assinaturas sincronizadas, então enviar a assinatura primeiro é o que destrava o relatório mais cedo.

Onde mostrar: completa, slide 22

Por que serviço e não método do agregado?

Porque a regra cruza coisas diferentes. A devolução, por exemplo, cruza o papel do usuário com o estado do ciclo; não pertence a nenhum dos dois sozinho.

Onde mostrar: completa, slides 20 a 22



Slide 10 de 27

Seção 2 — Dez casos de uso

50 s · no relógio: 7:10 → 8:00

Objetivo. Mostrar os dez casos de uso, um por arquivo, e o que cada um orquestra.

O QUE FALAR

A camada de aplicação tem a lógica de orquestração. Ela não tem regra de negócio e não tem código de interface.

São dez casos de uso, um por arquivo: registrar atividades, avaliar desempenho, autoavaliação, assinar, aprovar, devolver, gerar PDF, sincronizar fila, autenticar usuário e acessar via token.

A coluna “quem executa” mostra o papel. E a coluna do meio mostra que cada um chama o agregado ou um serviço de domínio: nenhum decide regra sozinho.

ONDE APONTAR

- Passe o cursor pela primeira coluna, de cima a baixo, enquanto fala os nomes.
- A última coluna: o arquivo de cada um.

SE O TEMPO ESTIVER CURTO

Mantenha, mas não leia a coluna do meio.

FRASE DE PASSAGEM

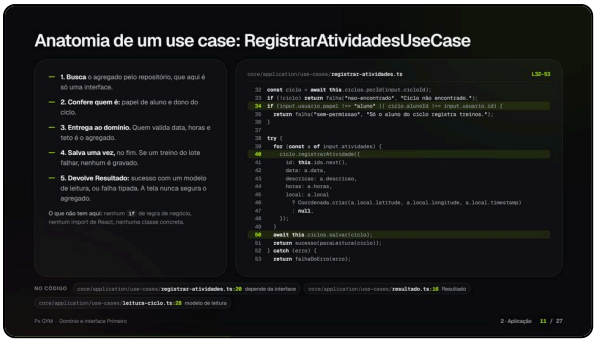
“Vou abrir um deles para mostrar que todos têm a mesma forma.”

PERGUNTAS QUE PODEM VIR AQUI

O que são os casos de uso “de apoio”?

Restaurar e encerrar sessão, e listar e consultar relatórios. Não entram na conta dos dez, mas são necessários para as telas funcionarem.

Onde mostrar: nota do slide



Slide 11 de 27

Anatomia de um use case

60 s · no relógio: 8:00 → 9:00

Objetivo. Mostrar, em um exemplo, os cinco passos que todos os casos de uso seguem.

O QUE FALAR

Este é o RegistrarAtividadesUseCase, e todos os outros seguem a mesma forma, em cinco passos.

Um: busca o agregado pelo repositório, que aqui é só uma interface. Dois: confere quem é o usuário, papel de aluno e dono do ciclo. Três: entrega ao domínio; quem valida data, horas e teto é o agregado. Quatro: salva uma vez, no fim, então se um treino do lote falhar nenhum é gravado. Cinco: devolve um Resultado, que é sucesso com um modelo de leitura ou falha tipada.

O que não tem aqui: nenhum if de regra de negócio, nenhum import de React e nenhuma classe concreta.

ONDE APONTAR

- As três linhas destacadas, na ordem: a checagem de papel, a chamada ao agregado e o salvar.

SE O TEMPO ESTIVER CURTO

Mantenha. É o slide que prova “orquestra, não decide”.

FRASE DE PASSAGEM

“O passo dois, a permissão, existe em todos. Juntei num quadro só.”

PERGUNTAS QUE PODEM VIR AQUI

O que é esse “modelo de leitura”?

É um objeto só de dados, que a tela recebe. A tela nunca segura a instância do agregado, então não tem como chamar uma regra por fora de um caso de uso.

Onde mostrar: completa, slide 36

Por que devolver Resultado em vez de lançar exceção?

Para o erro do domínio não vazar como exceção para a interface. O hook lê se deu certo e mostra a mensagem; nenhuma tela tem try/catch.

Onde mostrar: completa, slide 27



Slide 12 de 27

Quem pode o quê

45 s · no relógio: 9:00 → 9:45

Objetivo. Mostrar o controle de permissão por papel e os tipos de falha.

O QUE FALAR

Este quadro mostra quem pode fazer o quê, e a última coluna mostra a linha do código que decide.

O aluno registra treinos, se autoavalia e assina. O personal avalia e assina. A coordenação aprova e devolve.

E não basta o papel: o caso de uso também compara o id do usuário com o dono do relatório. Tem teste em que outro aluno tenta registrar treino no ciclo alheio e é barrado.

Toda recusa volta como falha tipada, com um destes cinco tipos, nunca como exceção.

ONDE APONTAR

- As bolinhas verdes por coluna.
- A coluna “linha que decide”.

SE O TEMPO ESTIVER CURTO

Mostre o quadro, diga “papel e dono do ciclo” e siga.

FRASE DE PASSAGEM

“E como tudo isso é testado sem banco? Com fakes em memória.”

PERGUNTAS QUE PODEM VIR AQUI

A permissão não deveria estar no backend?

Também vai estar, na fase de infraestrutura, com as políticas do banco. Aqui ela está na aplicação porque é regra de quem pode executar cada caso de uso, e precisa valer mesmo com tudo em memória.

Onde mostrar: slide 26 (limites)



Slide 13 de 27

Fakes em memória e os testes de use case

50 s · no relógio: 9:45 → 10:35

Objetivo. Mostrar o repositório fake em Map e os três tipos de teste: caminho feliz, erro e falta de permissão.

O QUE FALAR

O repositório fake usa um Map para simular o banco. Está aqui, junto com os outros fakes: sessão, PDF, sincronização, câmera, GPS, relógio.

Um detalhe: o Map guarda as props, não a instância. Cada leitura devolve um agregado novo, como um banco de verdade faria. Alterar o objeto sem salvar não vaza para o Map.

À direita, um teste de erro: um treino fora do ciclo no meio do lote. O resultado é falha de regra de negócio, e a linha destacada confere que o repositório não foi salvo nenhuma vez.

Cada caso de uso tem os três tipos de teste: caminho feliz, erro e falta de permissão.

ONDE APONTAR

- A primeira linha da tabela (CicloAvaliacaoRepositoryFake).
- A linha destacada do teste: salvamentos igual a zero.

SE O TEMPO ESTIVER CURTO

Diga “fake em Map” e “feliz, erro e permissão”, aponte a linha do teste.

FRASE DE PASSAGEM

“Domínio e aplicação são o core. Agora a camada que liga isso ao React Native.”

PERGUNTAS QUE PODEM VIR AQUI

Qual a diferença entre fake e mock?

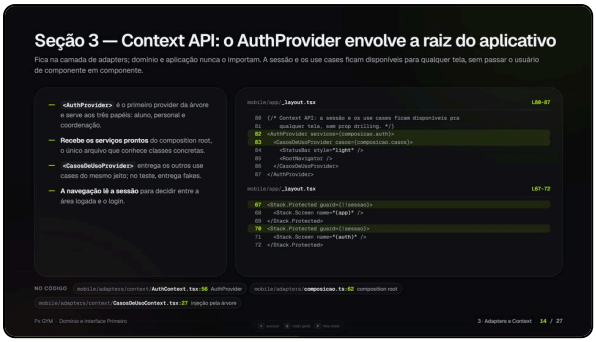
O fake é uma implementação de verdade da interface, só que simples, em memória. O mock é um objeto que registra chamadas. No core eu uso fakes; mock de módulo só aparece na borda, para as bibliotecas nativas do Expo.

Onde mostrar: completa, slides 37 e 50

Como você monta um ciclo aprovado no teste?

Pelo próprio domínio: o construtor de cenário abre o ciclo, registra, avalia, assina e aprova de verdade. Se uma regra mudar, o cenário quebra junto.

Onde mostrar: completa, slide 41



Slide 14 de 27

Seção 3 — Context API: o AuthProvider envolve a raiz

50 s · no relógio: 10:35 → 11:25

Objetivo. Responder três pontos sobre a Context API: onde fica, o que resolve e quem atende.

O QUE FALAR

A Context API fica na camada de adapters. O domínio e a aplicação nunca a importam.

O AuthProvider envolve a raiz do aplicativo. No código da direita ele é o primeiro provider da árvore. Com isso o estado da sessão fica disponível globalmente, e qualquer tela lê o usuário logado sem prop drilling. Vale para os três papéis: aluno, personal e coordenação.

O provider recebe os serviços prontos de um lugar só, o composition root, que é o único arquivo que conhece classes concretas.

Embaixo, a navegação lê a sessão para decidir entre a área logada e o login.

ONDE APONTAR

- A linha destacada do AuthProvider.
- As duas linhas com guard no segundo bloco.

SE O TEMPO ESTIVER CURTO

Mantenha. Context API é um dos pontos centrais da fase.

FRASE DE PASSAGEM

“Por dentro, o provider faz duas coisas.”

PERGUNTAS QUE PODEM VIR AQUI

Por que Context API e não Redux ou Zustand?

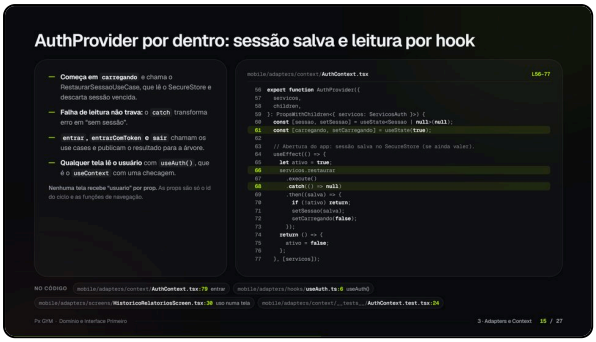
Porque o estado global aqui é pequeno: a sessão e a referência aos casos de uso. A Context API resolve o prop drilling sem dependência extra.

Onde mostrar: completa, slides 44 e 45

O que é o composition root?

É a função que dá new em tudo: monta cada caso de uso com suas dependências. Nesta fase ele monta repositórios em Map. Trocar por banco, na próxima fase, é mexer nesse arquivo e em nenhum caso de uso.

Onde mostrar: completa, slide 43



Slide 15 de 27

AuthProvider por dentro

45 s · no relógio: 11:25 → 12:10

Objetivo. Mostrar o carregamento da sessão salva e a leitura por hook.

O QUE FALAR

Quando o app abre, o provider começa em “carregando” e chama o caso de uso de restaurar sessão, que lê o SecureStore e descarta sessão vencida.

Na linha destacada do meio, se a leitura falhar, o catch transforma o erro em “sem sessão”. O app não trava na tela de carregamento.

E a leitura é por hook: qualquer tela chama useAuth, que é o useContext com uma checagem. Nenhuma tela recebe o usuário por prop.

ONDE APONTAR

- As três linhas destacadas: o estado inicial, a chamada ao restaurar e o catch.

SE O TEMPO ESTIVER CURTO

Pode pular: diga na seção anterior “ao abrir, ele restaura a sessão salva”.

FRASE DE PASSAGEM

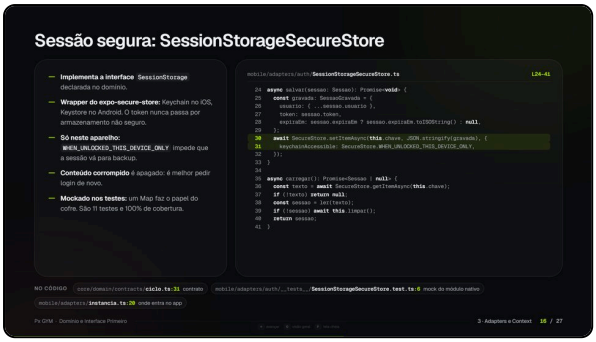
“Essa sessão precisa ficar guardada em algum lugar. Esse lugar é seguro.”

PERGUNTAS QUE PODEM VIR AQUI

Isso é testado?

Sim, três testes só para esse trecho: com sessão salva, sem sessão e com o SecureStore falhando. O arquivo do contexto tem dez testes.

Onde mostrar: completa, slides 46 e 75



Slide 16 de 27

Sessão segura: SessionStorageSecureStore

45 s · no relógio: 12:10 → 12:55

Objetivo. Mostrar o wrapper do expo-secure-store e por que ele é seguro.

O QUE FALAR

A sessão é guardada pela classe SessionStorageSecureStore, que implementa a interface SessionStorage declarada no domínio.

Ela é um wrapper do expo-secure-store. O token fica criptografado pelo sistema: Keychain no iOS, Keystore no Android. Nunca passa por armazenamento não seguro.

Na linha destacada, a opção de só neste aparelho, desbloqueado: a sessão não vai para backup.

E é o único componente real desta fase. Nos testes ele é mockado: um Map faz o papel do cofre.

ONDE APONTAR

- As duas linhas destacadas: setItemAsync e keychainAccessible.

SE O TEMPO ESTIVER CURTO

Mantenha. É item explícito do checklist.

FRASE DE PASSAGEM

“Entre as telas e os casos de uso ficam os hooks.”

PERGUNTAS QUE PODEM VIR AQUI

Por que não AsyncStorage?

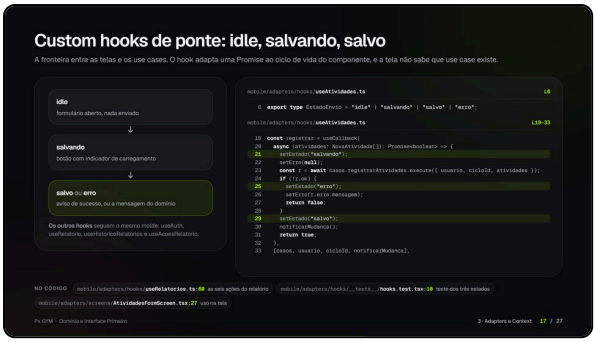
Porque o AsyncStorage guarda em texto puro. Token de sessão é credencial, então vai para o cofre do sistema operacional.

Onde mostrar: completa, slide 49

Se está tudo em mock, por que a sessão é real?

Porque sessão segura está no escopo da fase: é o último item do checklist. O que fica em mock é banco e hardware. E nos testes a sessão também é mockada.

Onde mostrar: slide 21



Slide 17 de 27

Custom hooks de ponte

45 s · no relógio: 12:55 → 13:40

Objetivo. Mostrar o hook adaptando o caso de uso ao ciclo de vida do componente, com os três estados: idle, salvando e salvo.

O QUE FALAR

Os hooks são a fronteira entre as telas e os casos de uso. O exemplo é o useAtividades.

Ele transforma a Promise do caso de uso em estados que a tela entende: idle, salvando e salvo, ou erro. No código, as três linhas destacadas são exatamente essas três mudanças de estado.

A tela não sabe que caso de uso existe. Ela só lê o estado e chama registrar.

Os outros hooks seguem o mesmo molde: useAuth, useRelatorio, useHistoricoRelatorios e useAcoesRelatorio.

ONDE APONTAR

- O fluxo vertical à esquerda.
- As três linhas destacadas à direita, de cima para baixo.

SE O TEMPO ESTIVER CURTO

Mostre o fluxo idle → salvando → salvo e siga.

FRASE DE PASSAGEM

“O último tipo de adapter é o de hardware.”

PERGUNTAS QUE PODEM VIR AQUI

Como testar o estado intermediário “salvando”?

Injetando um caso de uso fake que devolve uma Promise que o teste resolve na mão. Antes de resolver, o hook está em salvando; depois, em salvo.

Onde mostrar: completa, slide 76



Slide 18 de 27

Gateways de hardware

35 s · no relógio: 13:40 → 14:15

Objetivo. Mostrar o CameraGatewayExpo embrulhando o expo-camera atrás da interface do domínio.

O QUE FALAR

O CameraGatewayExpo embrulha a biblioteca nativa expo-camera atrás da interface CameraGateway, que foi declarada no domínio.

Ele traduz o vocabulário: a resposta do Expo vira concedida, negada ou bloqueada. A tela só desenha o preview; quem fala com a câmera é o gateway.

O GPS segue o mesmo padrão, com o LocationGatewayExpo, e entra na tela por prop: a rota entrega o do Expo e o teste entrega um fake.

ONDE APONTAR

- A primeira linha destacada: “implements CameraGateway”.

SE O TEMPO ESTIVER CURTO

Uma frase: “câmera e GPS ficam atrás de interfaces do domínio”.

FRASE DE PASSAGEM

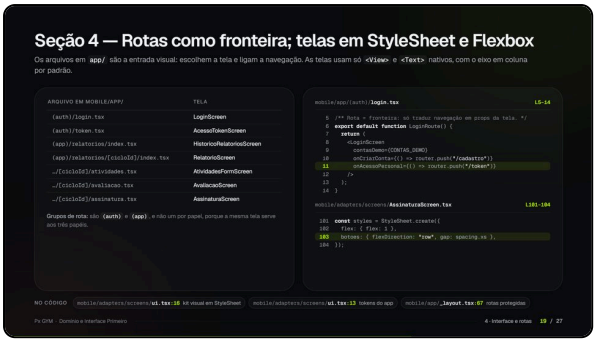
“Isso fecha os adapters. Agora o que o usuário vê.”

PERGUNTAS QUE PODEM VIR AQUI

A câmera é usada no relatório do ciclo?

Não. O gateway existe, é testado e é usado na rota de fotos do app. O relatório do ciclo usa o GPS, para o check-in do treino. Isso está no slide de limites.

Onde mostrar: slide 26



Slide 19 de 27

Seção 4 — Rotas e layout

40 s · no relógio: 14:15 → 14:55

Objetivo. Cobrir Expo Router como fronteira e o padrão de estilo: View e Text com StyleSheet e Flexbox.

O QUE FALAR

A pasta app, do Expo Router, é a fronteira de entrada visual: é onde o toque do usuário entra. Cada arquivo de rota só escolhe a tela e liga a navegação, como no exemplo do login à direita.

As telas são construídas com View e Text nativos, estilizadas com StyleSheet e Flexbox. O eixo principal é coluna, que é o padrão; linha só aparece onde é declarada, como nesse par de botões.

Sobre as pastas: usei os grupos auth e app, e não um grupo por papel, porque a mesma tela de relatório serve aos três papéis.

ONDE APONTAR

- A tabela de rotas.
- No código de baixo, a linha com flexDirection: "row".

SE O TEMPO ESTIVER CURTO

Mantenha em 20 segundos.

FRASE DE PASSAGEM

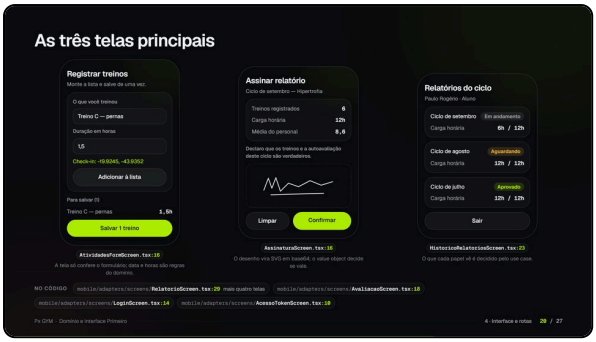
“E estas são as três telas principais.”

PERGUNTAS QUE PODEM VIR AQUI

O resto do app usa NativeWind. Por que aqui é StyleSheet?

Porque esta fase é feita com StyleSheet e Flexbox, os recursos nativos do React Native. As telas do relatório foram escritas assim, usando os mesmos tokens de cor, raio e espaçamento do design system do app.

Onde mostrar: completa, slides 59 e 60



Slide 20 de 27

As três telas principais

50 s · no relógio: 14:55 → 15:45

Objetivo. Mostrar AtividadesFormScreen, AssinaturaScreen e HistoricoRelatoriosScreen e a ideia de que a tela não tem regra.

O QUE FALAR

As três telas principais do relatório. Um aviso honesto: são desenhos feitos a partir do código, não capturas de aparelho.

No formulário de atividades o aluno monta uma lista de treinos e salva de uma vez. A tela só confere se o campo está preenchido; data fora do ciclo e teto de horas são regras do domínio e voltam como erro.

Na tela de assinatura o desenho vira um SVG em base64 e vai para o caso de uso. Quem decide se aquilo é uma assinatura de verdade é o value object.

E o histórico lista os relatórios. O que cada papel vê é decidido pelo caso de uso, não pela tela.

ONDE APONTAR

- Cada desenho, da esquerda para a direita, acompanhando a fala.

SE O TEMPO ESTIVER CURTO

Mantenha. É o único slide visual do app.

FRASE DE PASSAGEM

“Tudo isso roda sem banco. É a seção cinco.”

PERGUNTAS QUE PODEM VIR AQUI

Tem como ver rodando?

Nesta fase a validação foi por teste automatizado, incluindo um teste de ponta a ponta que percorre todas as telas. Não rodei em aparelho. Se o senhor quiser, eu mostro o teste rodando.

Onde mostrar: slide 24 e a demonstração do terminal

São só três telas?

São sete: além dessas, login, acesso por token, o relatório e a avaliação. Estão nos chips do rodapé.

Onde mostrar: completa, slide 64



Slide 21 de 27

Seção 5 — O que fica em mock ou fake

50 s · no relógio: 15:45 → 16:35

Objetivo. Responder item a item o que fica em mock nesta fase, com a prova.

O QUE FALAR

A seção cinco é o que fica em mock nesta fase. Esta tabela responde item a item.

Sem SQLite: no modo mock o arquivo do banco nem é aberto. Sem Supabase: o cliente não é criado. Câmera e GPS usam gateways fakes nos testes. Os repositórios são Map em memória. A autenticação é um stub com três contas fixas.

O modo mock é o padrão do aplicativo. E tem um teste que prova: ele carrega os módulos de banco e de nuvem e confere que a função que abre o SQLite e a que cria o cliente do Supabase não foram chamadas.

ONDE APONTAR

- As duas primeiras linhas (SQLite e Supabase).
- A nota de baixo: o teste que prova e as contas de demonstração.

SE O TEMPO ESTIVER CURTO

Mantenha. “100% mock” está no título da fase.

FRASE DE PASSAGEM

“Como nada depende de banco, a suíte roda em segundos. É a seção seis.”

PERGUNTAS QUE PODEM VIR AQUI

Mas o código de SQLite e Supabase existe no projeto?

Existe, da fase anterior, e está desligado. Uma variável de ambiente religa. No modo mock, quem tentar usar o banco recebe um erro na hora dizendo que ele está desligado.

Onde mostrar: completa, slides 66 a 68

Tem algo que não é mock?

Duas coisas, de propósito. A sessão segura grava de verdade no cofre do aparelho, porque está no escopo da fase. E, rodando num celular, o check-in usaria o GPS real; nos testes é fake.

Onde mostrar: slide 16



Slide 22 de 27

Seção 6 — A pirâmide de testes

50 s · no relógio: 16:35 → 17:25

Objetivo. Mostrar que a suíte tem a forma de uma pirâmide, com números reais.

O QUE FALAR

A pirâmide de testes tem cinco níveis, e estes são os números reais.

Na base, o domínio, com 176 testes puros, sem nenhum mock. Depois os casos de uso, com 105, usando fakes em memória. No meio, Context, hooks e os demais adaptadores, com 67. Telas e componentes, com 32. E no topo, um único teste de ponta a ponta, para o fluxo crítico.

As barras estão em escala linear. Só do ciclo de avaliação são 115 de domínio e 69 de casos de uso; o resto desses dois níveis é do app que já existia.

ONDE APONTAR

- As barras, de baixo para cima.
- O número 1 no topo: ponta a ponta é pouco, de propósito.

SE O TEMPO ESTIVER CURTO

Mantenha.

FRASE DE PASSAGEM

“E a meta de cobertura.”

PERGUNTAS QUE PODEM VIR AQUI

Por que juntou Context e hooks com adaptadores?

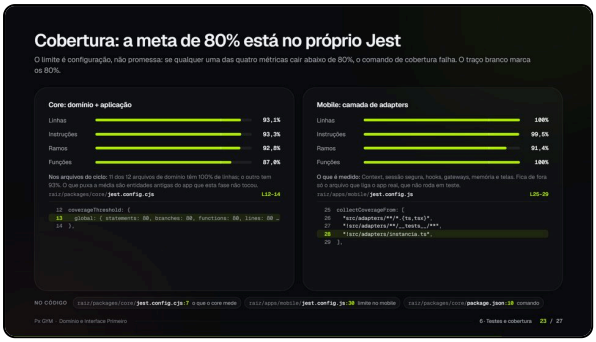
Context e hooks sozinhos são 20 testes. Os outros 47 são da mesma camada: sessão segura, gateways e repositórios em memória. Juntei porque estão no mesmo nível da pirâmide, e separei os números na legenda.

Onde mostrar: legenda do slide

Por que só um E2E?

Porque é caro e lento, e fica reservado para o fluxo crítico. Esse único teste percorre os três papéis, do primeiro treino ao PDF aprovado.

Onde mostrar: completa, slide 79



Slide 23 de 27

Cobertura: a meta de 80%

40 s · no relógio: 17:25 → 18:05

Objetivo. Mostrar os números de cobertura e que o limite de 80% é configuração do Jest.

O QUE FALAR

A meta de 80% não é promessa, é configuração. Está no arquivo do Jest: se qualquer uma das quatro métricas cair abaixo de 80%, o comando de cobertura falha.

No core, que é domínio mais aplicação, são 93% de linhas, e a menor métrica é funções, com 87%. Na camada de adapters do mobile são 100% de linhas e 91% de ramos.

O traço branco em cada barra marca os 80%.

ONDE APONTAR

- O traço branco numa das barras.
- O bloco de código com coverageThreshold.

SE O TEMPO ESTIVER CURTO

Diga os dois números e “o limite está no Jest”.

FRASE DE PASSAGEM

“E como é um teste de tela.”

PERGUNTAS QUE PODEM VIR AQUI

Por que funções está em 87%?

Porque a medição do core inclui entidades antigas do app, sem teste próprio, que esta fase não tocou. Nos doze arquivos de domínio do ciclo, onze têm 100% de linhas e um tem 93%.

Onde mostrar: nota do slide

O que fica de fora da medição?

No mobile, só o arquivo que liga o app real, com SecureStore e Supabase de verdade, que não roda em teste. Está escrito na configuração.

Onde mostrar: bloco de código da direita



Slide 24 de 27

Testes de tela com RNTL e o E2E

50 s · no relógio: 18:05 → 18:55

Objetivo. Mostrar render, fireEvent.press e fireEvent.changeText num teste real, e descrever o E2E.

O QUE FALAR

Os testes de tela usam a React Native Testing Library com jest-expo. Este é o teste do formulário de atividades.

A tela é renderizada dentro do AuthProvider, com um aluno logado. O changeText preenche os campos, encontrados pelo rótulo. O press toca nos botões, encontrados por papel e nome. O GPS é um fake injetado por prop.

O que o teste confere é o que o usuário vê: os textos, o contador e o botão desabilitado.

E o teste de ponta a ponta percorre os três papéis: o aluno registra e assina, o personal entra por token e avalia, a coordenação aprova, sincroniza e gera o PDF.

ONDE APONTAR

- As três linhas destacadas: a montagem com aluno logado e os dois press.

SE O TEMPO ESTIVER CURTO

Mostre as três palavras: render, press, changeText.

FRASE DE PASSAGEM

“Para fechar, o checklist da fase.”

PERGUNTAS QUE PODEM VIR AQUI

Como a tela recebe casos de uso fakes?

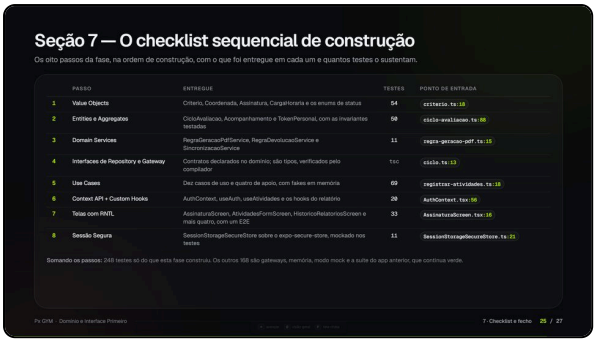
Pelo contexto. Um auxiliar de teste monta a mesma árvore de providers do app e deixa trocar qualquer caso de uso por um fake. Não precisa mockar módulo.

Onde mostrar: completa, slide 78

Pode rodar agora?

Posso. No terminal, dentro de apps/mobile, npm test. Leva cerca de meio minuto.

Onde mostrar: seção “Demonstração ao vivo” deste roteiro



Slide 25 de 27

Seção 7 — O checklist sequencial

45 s · no relógio: 18:55 → 19:40

Objetivo. Mostrar os oito passos da fase como entregues, com número de testes.

O QUE FALAR

A fase tem um checklist de oito passos, nesta ordem de construção. Todos foram entregues.

Value objects, com 54 testes. Entidades e agregados, com 50. Serviços de domínio, com 11. As interfaces, que são tipos e são verificadas pelo compilador. Os casos de uso, com 69. Context API e hooks, com 20. Telas com RNTL, com 33 contando o E2E. E a sessão segura, com 11.

Somando, são 248 testes só do que esta fase construiu.

ONDE APONTAR

- A coluna de números, de cima para baixo.
- A última coluna: o ponto de entrada no código de cada passo.

SE O TEMPO ESTIVER CURTO

Mantenha. É o placar final da fase.

FRASE DE PASSAGEM

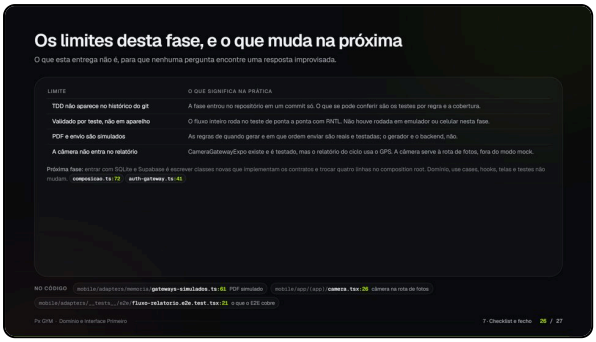
“Antes de concluir, quero ser franco sobre o que esta entrega não é.”

PERGUNTAS QUE PODEM VIR AQUI

O passo 4 não tem teste?

Interfaces são tipos: não existem em tempo de execução, então não há o que executar. Quem verifica é o compilador, e os fakes e adapters que as implementam é que são testados.

Onde mostrar: slide 9



Slide 26 de 27

Os limites desta fase

50 s · no relógio: 19:40 → 20:30

Objetivo. Dizer os limites antes que perguntem. Isso aumenta a credibilidade de todo o resto.

O QUE FALAR

Quatro limites, ditos com franqueza.

O TDD não aparece no histórico do git, porque a fase entrou no repositório em um commit só; o que dá para conferir são os testes por regra e a cobertura. A validação foi por teste automatizado, não em aparelho. O PDF e o envio são simulados: as regras de quando gerar e em que ordem enviar são reais, o gerador e o backend não. E a câmera não entra no relatório do ciclo.

E a próxima fase: entrar com SQLite e Supabase é escrever classes novas que implementam os contratos e trocar quatro linhas no composition root. Domínio, casos de uso, hooks, telas e testes não mudam.

ONDE APONTAR

- A linha do TDD e a linha “validado por teste”. São as duas mais prováveis de virar pergunta.

SE O TEMPO ESTIVER CURTO

Não corte. Dizer os limites primeiro é melhor do que ser perguntado.

FRASE DE PASSAGEM

“Resumindo o que foi entregue.”

PERGUNTAS QUE PODEM VIR AQUI

Então você não fez TDD?

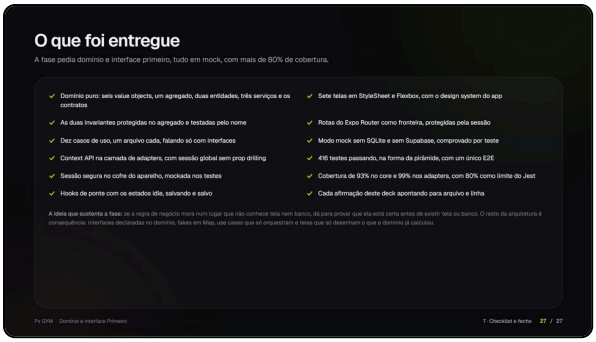
Fiz o ciclo teste primeiro, código depois, mas não fiz commits separados para cada passo, então o histórico não prova isso. O que eu consigo mostrar é que cada regra tem um teste com o nome dela e que a cobertura passa de 90%. Na próxima fase vou fazer commits menores.

Onde mostrar: completa, slide 80

Por que não testou no celular?

Nesta fase priorizei a suíte automatizada, que é o foco desta fase. O teste de ponta a ponta roda o aplicativo inteiro, menos a navegação nativa. Rodar em aparelho é o primeiro passo da próxima fase.

Onde mostrar: completa, slide 79



Slide 27 de 27

O que foi entregue

40 s · no relógio: 20:30 → 21:10

Objetivo. Fechar com a ideia central e abrir para perguntas.

O QUE FALAR

Em resumo: domínio puro com as invariantes protegidas; dez casos de uso falando só com interfaces; Context API e sessão segura na camada de adapters; sete telas em StyleSheet e Flexbox; modo mock sem banco, comprovado por teste; e 416 testes na forma da pirâmide, com 93% e 99% de cobertura.

A ideia que sustenta a fase é esta: se a regra de negócio mora num lugar que não conhece tela nem banco, dá para provar que ela está certa antes de existir tela ou banco.

Obrigado. Fico à disposição para perguntas, e posso abrir qualquer arquivo que o senhor quiser ver.

ONDE APONTAR

- Não leia a lista. Diga o resumo de memória e termine na frase final, olhando para a banca.

SE O TEMPO ESTIVER CURTO

Mantenha. Termine sempre na frase da ideia central.

Parte 3 — Perguntas difíceis

Além das perguntas listadas em cada slide, estas são as que testam se você entende o que apresentou. As respostas estão no tamanho certo para falar: duas ou três frases.

Regras para responder qualquer pergunta

- **Responda primeiro, explique depois.** Comece por “sim”, “não” ou o dado pedido.
- **Mostre onde está.** Sempre que der, vá ao slide ou abra o arquivo. É o que o professor pediu.
- **Se não souber, diga.** “Não sei responder isso agora; posso verificar no código e trazer.” É muito melhor do que inventar.
- **Se a pergunta for sobre um limite, concorde.** Você já disse os limites no slide 26; a resposta é “sim, e está previsto para a próxima fase”.

Conceitos

O que é uma invariante?

É uma condição que precisa ser verdadeira sempre, não importa a ordem das operações. Aqui: um ciclo aprovado nunca recebe uma segunda avaliação, e o PDF nunca é gerado sem todas as assinaturas sincronizadas. Quem garante é o agregado, porque é o único lugar por onde o estado muda.

Mostrar: slide 7.

O que é um aggregate root e por que tudo passa por ele?

É a entidade que serve de porta de entrada para um conjunto de objetos que precisam ser consistentes entre si. Atividades, avaliações e assinaturas só mudam por método do CicloAvaliacao. Se alguém pudesse alterar a lista de assinaturas por fora, a regra do PDF deixaria de valer.

Mostrar: slide 6.

Qual a diferença entre serviço de domínio e caso de uso?

O serviço de domínio guarda regra de negócio que não cabe em uma entidade só, e não fala com o mundo externo. O caso de uso não tem regra: ele busca no repositório, chama o domínio, salva e devolve o resultado. Um decide, o outro orquestra.

Mostrar: slides 9 e 11.

Por que as interfaces ficam no domínio e não junto das implementações?

É a inversão de dependência. Quem define o contrato é quem precisa dele. Assim o domínio não depende de SQLite nem de Expo; são eles que dependem do domínio, implementando a interface.

Mostrar: slide 9; completa, slide 23.

Fake, stub e mock são a mesma coisa?

Não. O fake é uma implementação simples que funciona de verdade, como o repositório em Map. O stub devolve respostas fixas, como a autenticação com três contas. O mock substitui um módulo e registra chamadas, e eu uso só para as bibliotecas nativas do Expo.

Mostrar: slides 13 e 21.

Por que um hook entre a tela e o caso de uso? A tela não podia chamar direto?

Poderia, mas a tela ficaria cuidando de Promise, estado de carregamento e mensagem de erro. O hook adapta o caso de uso ao ciclo de vida do React e entrega só o estado pronto. A tela fica só com o desenho, e isso também a deixa fácil de testar.

Mostrar: slide 17.

O que é prop drilling e como a Context API resolve?

É passar um dado por vários níveis de componentes que não o usam, só para chegar em quem precisa. Com o AuthProvider na raiz, qualquer tela lê o usuário com useAuth, sem que os componentes do meio saibam que ele existe.

Mostrar: slides 14 e 15.

Decisões do projeto

O que acontece quando você trocar o Map por SQLite e Supabase?

Escrevo classes novas que implementam os mesmos contratos e troco a montagem no composition root. Domínio, casos de uso, hooks e telas não mudam, e os testes de caso de uso continuam usando os fakes.

Mostrar: slide 26; completa, slide 86.

Onde fica a validação: na tela ou no domínio?

A tela só confere o formulário: campo vazio e texto que não é número. Regra de negócio, como data fora do ciclo ou teto de horas, é do domínio e volta para a tela como mensagem de erro.

Mostrar: slide 20; completa, slide 61.

Como funciona a sincronização, se não tem backend?

Cada treino, avaliação e assinatura nasce com status pending. O caso de uso de sincronizar monta a fila na ordem que o domínio define e marca cada item como synced ou error. Nesta fase o envio é simulado; o que é real e testado é a fila, a ordem e a marcação.

Mostrar: slide 9; completa, slides 22 e 33.

O token do personal é seguro?

Ele tem no mínimo oito caracteres, validade de uma hora a trinta dias e pode ser revogado. A sessão criada expira junto com ele. Nesta fase os tokens vivem em memória; guardar e validar no servidor é da fase de infraestrutura.

Mostrar: slide 8; completa, slides 19 e 35.

Cobertura alta garante que o código está certo?

Não. Cobertura mostra o que foi executado, não o que foi verificado. Por isso os testes são escritos por regra, com o nome da regra, e não para “pintar” linhas. A cobertura é a consequência.

Mostrar: slide 23; completa, slide 80.

Quanto tempo a suíte leva?

O core roda em cerca de meio minuto e o mobile entre dez segundos e um minuto, sem emulador e sem rede. É rápido justamente porque nada depende de banco ou de hardware.

Mostrar: terminal.

Perguntas delicadas

Você fez TDD de verdade?

Segui o ciclo de escrever o teste da regra e depois o código, mas a fase entrou no repositório em um commit só, então o histórico não prova isso. O que posso mostrar é que cada regra tem um teste com o nome dela. Na próxima fase vou registrar em commits menores.

Mostrar: slide 26.

Isso roda no celular?

O fluxo foi validado por testes automatizados, incluindo um de ponta a ponta. Não rodei em aparelho nesta fase, e não quero afirmar o que não verifiquei. É o primeiro passo da próxima.

Mostrar: slides 24 e 26.

Você usou inteligência artificial?

Responda a verdade, conforme a regra da disciplina. O histórico do git registra coautoria de assistente de IA nos commits, então negar não é uma opção. O que sustenta a sua nota é você conseguir explicar qualquer arquivo: por isso vale estudar os slides 6, 7 e 11 até conseguir reescrever aquela lógica de cabeça.

Preparação: abra os três arquivos e leia linha a linha antes do dia.

Pode mostrar um arquivo que não está em nenhum slide?

Sim. Abra o editor e vá ao arquivo. Se não lembrar onde fica, a versão completa tem dois slides de rastreabilidade, o 83 e o 84, que ligam cada requisito da fase ao arquivo e ao teste.

Mostrar: completa, slides 83 e 84.

Parte 4 — Consulta rápida

Mapa: onde está cada coisa

“Curto” é o deck de 27 slides que você apresenta. “Completa” é o de 87, na segunda aba, com o código de cada item.

SE PERGUNTAREM SOBRE	CURTO	COMPLETA
Criterio, Coordenada, Assinatura, CargaHoraria (código de cada um)	5	7–10
Enums de status e diagrama do ciclo de vida	5, 6	11
Imutabilidade e igualdade por valor	5	12
Agregado e suas invariantes	6, 7	13–17
Acompanhamento e TokenPersonal	8	18, 19
Os três serviços de domínio (código)	9	20–22
Contratos e papéis	9	23, 24
Lista dos casos de uso	10	25
Resultado e tipos de falha	12	27
Código de cada caso de uso	11	26, 29–35
Modelo de leitura	11	36
Fakes e testes de caso de uso	13	37–41
Composition root	14	43
AuthContext por dentro	14, 15	44–48
Sessão segura e o mock nos testes	16	49, 50
Hooks	17	51–54
Gateways de câmera e GPS	18	55, 56
Rotas, layout e kit visual	19	57–60
Cada tela, com código	20	61–64
Modo mock em detalhe	21	65–70
Pirâmide e cobertura	22, 23	71, 72
Testes por nível, com código	24	73–79
TDD: regra, teste e código lado a lado	–	80
Como rodar os testes	–	81
Rastreabilidade item a item	25	83, 84
Limites e próxima fase	26	85, 86

Os números, para ter na ponta da língua

NÚMERO	O QUE É
416	testes passando no total: 281 no core, 115 no mobile, 20 no pacote de banco
248	testes só do que esta fase construiu (soma dos oito passos do checklist)
176 · 105 · 67 · 32 · 1	pirâmide: domínio, casos de uso, Context/hooks/adaptadores, telas, ponta a ponta
115 e 69	testes de domínio e de casos de uso só do ciclo de avaliação
93%	cobertura de linhas do core; a menor métrica é funções, com 87%
99%	cobertura dos adapters (100% de linhas, 91% de ramos)
80%	limite configurado no Jest para as quatro métricas
6 · 1 · 2 · 3	value objects, agregado, entidades, serviços de domínio
10	casos de uso (mais 4 de apoio)
7	telas em StyleSheet e Flexbox

Contas de demonstração

Só servem se você abrir o app. No modo mock: `aluno@pxgym.dev`, `personal@pxgym.dev` e `coordenacao@pxgym.dev`, todas com a senha `pxgym123`. O personal sem conta entra com o código `CARLA-2026`.

Glossário de bolso

Value object. Tipo imutável, sem identidade, validado na criação. Dois com os mesmos valores são iguais.

Entidade. Objeto com identidade própria, que continua sendo o mesmo ainda que os dados mudem.

Aggregate root. Entidade que é a única porta de entrada para um conjunto de objetos que precisam ficar consistentes.

Invariante. Regra que tem de ser verdadeira sempre, em qualquer ordem de operações.

Serviço de domínio. Regra de negócio que cruza entidades e não cabe em nenhuma delas.

Caso de uso. Orquestra uma ação do usuário: busca, chama o domínio, salva, responde.

Contrato ou port. Interface que o domínio declara para algo externo: repositório, câmera, GPS, autenticação.

Adapter. Implementação de um contrato com uma tecnologia concreta, ou código que liga o React ao core.

Gateway. Adapter para um recurso externo, como hardware ou serviço de rede.

Repositório. Contrato para guardar e recuperar agregados.

Composition root. O único lugar que instancia classes concretas e monta as dependências.

Context API. Mecanismo do React para disponibilizar um valor a toda a árvore de componentes.

Prop drilling. Passar um dado por vários níveis de componentes que não o usam.

Fake. Implementação simples e funcional de um contrato, em memória.

Stub. Substituto que devolve respostas fixas.

Mock. Substituto de um módulo que registra como foi chamado.

RNTL. React Native Testing Library: renderiza a tela e simula toques e digitação.

E2E. Teste de ponta a ponta, que percorre um fluxo completo do usuário.

Modelo de leitura. Objeto só de dados que o caso de uso entrega à tela, no lugar do agregado.

Offline-first. O dado nasce no aparelho e é enviado depois; daí os status pending, synced e error.

O que não dizer

EVITE	DIGA
“Cem por cento de cobertura.”	“93% de linhas no core e 99% nos adapters.”
“Testei no celular.”	“Validei por testes automatizados, incluindo um de ponta a ponta.”
“O histórico do git mostra o TDD.”	“Cada regra tem um teste com o nome dela; o histórico tem um commit só.”
“Não tem nenhum código de banco no projeto.”	“O código de banco da fase anterior existe e está desligado no modo mock.”
“O PDF é gerado.”	“A regra de quando gerar é real; o gerador é simulado nesta fase.”
“Isso aqui é complicado de explicar.”	Explique com o exemplo do slide; todos foram escolhidos para isso.
“Como vocês podem ver…” e ler o slide	Diga o que o slide prova e aponte uma linha.

Se algo der errado

- **O deck não abre ou aparece sem fonte:** abra o PDF de reserva que você gerou na véspera.
- **O projetor corta as bordas:** saia da tela cheia (`F`) e reduza a janela; o deck se reajusta.
- **Trocou de slide sem querer ao clicar:** `←` volta. A partir daí, aponte sem clicar.
- **Perdeu o fio:** diga a frase de passagem do slide em que está, que está na Parte 2, e siga.
- **Deu branco numa pergunta:** “vou mostrar no código”. Abrir o arquivo dá tempo de pensar e é exatamente o que foi pedido.

Checklist final

- [ ] Li este roteiro inteiro e ensaiei em voz alta com cronômetro duas vezes.
- [ ] Sei de cor a frase de abertura, as duas invariantes e a frase final.
- [ ] Consigo explicar, sem olhar, os slides 6, 7 e 11.
- [ ] Abri os arquivos `ciclo-avaliacao.ts`, `registrar-atividades.ts` e `AuthContext.tsx` e li cada um.
- [ ] Rodei os testes hoje e estão verdes.
- [ ] Os dois decks estão abertos, a partir de `C:\dev\Px GYM`, e o PDF de reserva está salvo.
- [ ] Editor e terminal estão abertos na pasta do projeto, com fonte grande.
- [ ] Notificações desligadas, notebook na tomada, cronômetro à vista.